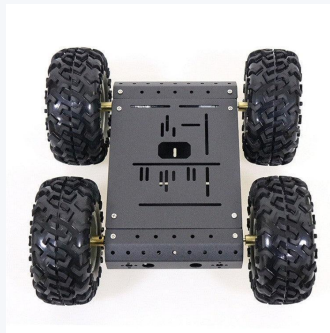


ESP32 & Pont H

Robot Tout-Terrain Contrôlé par Manette PS4



Robot Tout-Terrain



Plateforme : ESP32 (Xtensa LX6 double cœur, 240 MHz)
Driver moteur : Double pont H (L298N ou équivalent)
Contrôleur : Sony DualShock 4 (Bluetooth)
Bibliothèque : PS4-esp32 par *aed3*
Date : 2 juin 2026

Table des matières

1	Introduction	2
2	Présentation du matériel	2
2.1	Microcontrôleur ESP32	2
2.2	Driver pont H	2
2.3	Manette PS4 DualShock 4	2
3	Plateforme Robot Tout-Terrain	3
4	Architecture logicielle	3
4.1	Bibliothèque requise	3
4.2	Définitions des broches et constantes	3
4.3	Fonctions de mouvement	3
4.4	Boucle principale	4
5	Montage et câblage	4
5.1	Schéma de câblage	4
5.2	Association de la manette PS4	5
6	Résultats et discussion	5
7	Conclusion	5

1. Introduction

Ce rapport présente la conception et la mise en œuvre d'un **robot tout-terrain contrôlé sans fil** basé sur le microcontrôleur **ESP32**. Le système utilise un double pont H pour commander deux moteurs à courant continu et une manette **Sony DualShock 4 (PS4)** comme interface homme-machine.

L'ESP32 intègre le Wi-Fi et le Bluetooth Classic dans un SoC double cœur à 240 MHz, ce qui en fait une plateforme idéale pour les applications de robotique embarquée. La bibliothèque **PS4-esp32** (par *aed3*) permet d'associer le microcontrôleur à une manette PS4 via Bluetooth et de lire ses entrées directionnelles en temps réel.

Objectifs principaux

- Contrôle sans fil complet d'un robot à entraînement différentiel deux moteurs
- Commandes de direction en temps réel : avant, arrière, gauche, droite, stop
- Firmware propre et maintenable avec abstraction des mouvements par fonctions
- Initialisation sécurisée (moteurs à l'arrêt à la mise sous tension)

2. Présentation du matériel

2.1. Microcontrôleur ESP32

L'**ESP32** (Espressif Systems) est un SoC basse consommation et bas coût qui intègre :

- Un processeur double cœur Xtensa LX6 jusqu'à 240 MHz
- Bluetooth Classic (BR/EDR) et BLE intégrés
- 34 broches GPIO programmables (PWM, ADC, DAC, SPI, I²C, UART)
- 520 Ko de SRAM, 4 Mo de Flash (module typique)

2.2. Driver pont H

Un double pont H (ex. **L298N**) pilote indépendamment les deux moteurs DC. Chaque canal nécessite deux broches de direction et une broche PWM d'activation :

Signal	GPIO ESP32	Macro	Rôle
Moteur droit IN1	GPIO 14	rm1	Direction A
Moteur droit IN2	GPIO 27	rm2	Direction B
Moteur gauche IN1	GPIO 12	lm1	Direction A
Moteur gauche IN2	GPIO 13	lm2	Direction B
Enable moteur droit	GPIO 25	v1	Vitesse PWM
Enable moteur gauche	GPIO 26	v2	Vitesse PWM

2.3. Manette PS4 DualShock 4

La Sony DualShock 4 se connecte à l'ESP32 via **Bluetooth Classic (BR/EDR)**. L'association initiale requiert l'écriture de l'adresse MAC de l'ESP32 dans la manette à l'aide d'un outil PC (ex. *SixaxisPairTool*). L'adresse MAC est définie dans le firmware par la constante `MPU_MAC`.

3. Plateforme Robot Tout-Terrain

Les robots tout-terrain sont des véhicules à roues ou à chenilles conçus pour évoluer sur des terrains accidentés, irréguliers ou encombrés d'obstacles. Ils sont largement utilisés dans les domaines suivants :

- Opérations de recherche et sauvetage
- Inspection et surveillance agricole
- Reconnaissance militaire
- Robotique éducative et amateur

La plateforme utilisée dans ce projet est un **châssis à entraînement différentiel** équipé de roues de grand diamètre adaptées aux terrains difficiles. Les deux moteurs DC sont pilotés indépendamment, permettant des rotations sur place et une navigation précise.

4. Architecture logicielle

4.1. Bibliothèque requise

Installer la bibliothèque **PS4Controller** dans l'IDE Arduino :

Arduino IDE → Gérer les bibliothèques → rechercher "PS4Controller"
 GitHub : <https://github.com/aed3/PS4-esp32>

4.2. Définitions des broches et constantes

```

1 #include <PS4Controller.h>
2
3 #define rm1 14 // Moteur droit direction A
4 #define rm2 27 // Moteur droit direction B
5 #define lm1 12 // Moteur gauche direction A
6 #define lm2 13 // Moteur gauche direction B
7 #define v1 25 // Enable moteur droit (PWM)
8 #define v2 26 // Enable moteur gauche (PWM)
9
10 #define MPU_MAC "e8:61:7e:58:16:ce" // Adresse MAC Bluetooth ESP32
11 #define SPEED_MAX 255 // Vitesse maximale (0-255)

```

Listing 1 – Définitions des broches et constantes

4.3. Fonctions de mouvement

Cinq fonctions abstraient tous les primitives de mouvement :

```

1 void AVANT_MAX() {
2     digitalWrite(rm1, HIGH); digitalWrite(rm2, LOW);
3     digitalWrite(lm1, HIGH); digitalWrite(lm2, LOW);
4     analogWrite(v1, SPEED_MAX);
5     analogWrite(v2, SPEED_MAX);
6 }

```

Listing 2 – Fonction de mouvement — AVANT_MAX

```

1 void STOP() {
2     digitalWrite(rm1, LOW);  digitalWrite(rm2, LOW);
3     digitalWrite(lm1, LOW);  digitalWrite(lm2, LOW);
4     analogWrite(v1, 0);
5     analogWrite(v2, 0);
6 }

```

Listing 3 – Fonction d’arrêt — STOP

Le tableau ci-dessous récapitule la logique des cinq fonctions :

Fonction	rm1	rm2	lm1	lm2	v1	v2
AVANT_MAX	H	L	H	L	255	255
ARRIERE_MAX	L	H	L	H	255	255
DROITE_MAX	H	L	L	H	255	255
GAUCHE_MAX	L	H	H	L	255	255
STOP	L	L	L	L	0	0

4.4. Boucle principale

```

1 void loop() {
2     if (PS4.isConnected()) {
3         if (PS4.Up()) { AVANT_MAX(); Serial.println("avant");
4             }
5         else if (PS4.Down()) { ARRIERE_MAX(); Serial.println("arriere");
6             }
7         else if (PS4.Left()) { DROITE_MAX(); Serial.println("droite");
8             }
9         else if (PS4.Right()) { GAUCHE_MAX(); Serial.println("gauche");
10            }
11        else { STOP(); Serial.println("stop"); }
12    } else {
13        STOP(); // Securite : moteurs arretes si manette deconnectee
14    }
15 }

```

Listing 4 – Boucle de contrôle principale

Remarque : Le mappage des boutons Gauche/Droite du D-pad est intentionnellement inversé pour correspondre à l’orientation physique du robot par rapport à la manette.

5. Montage et câblage

5.1. Schéma de câblage

Le câblage suit une configuration double pont H :

- Chaque canal du pont H contrôle un moteur via deux broches de direction et une activation PWM.
- La logique 3,3 V de l’ESP32 est compatible avec la plupart des modules L298N.
- Une **alimentation séparée** (7–12 V) est obligatoire pour les moteurs ; n’alimentez **jamais** les moteurs depuis les rails 3,3 V ou 5 V de l’ESP32.

5.2. Association de la manette PS4

1. Lire l'adresse MAC Bluetooth de l'ESP32 (affichée dans `setup()` via le port Série).
2. Sur un PC Windows/Linux, utiliser **SixaxisPairTool** pour écrire cette adresse dans la manette PS4.
3. Mettre l'ESP32 sous tension, puis maintenir SHARE + PS sur la manette pour entrer en mode d'association.
4. La manette se connecte et le robot est opérationnel.

6. Résultats et discussion

Le système mis en œuvre permet :

- Un **contrôle sans fil fiable** jusqu'à environ 10 m en visibilité directe
- Une **faible latence** de commande (<50 ms de délai perçu)
- Un **comportement sécurisé** lors de la déconnexion de la manette (arrêt automatique des moteurs)
- Une **extensibilité** : la vitesse peut être modulée en remplaçant `SPEED_MAX` par une valeur PWM dérivée des joysticks analogiques (`PS4.LStickY()`)

Améliorations possibles :

- Contrôle de vitesse variable via les joysticks analogiques
- Ajout de capteurs ultrasoniques pour l'évitement d'obstacles
- Intégration d'une centrale inertielle MPU6050 pour le retour d'orientation
- Mise à jour du firmware en OTA via Wi-Fi

7. Conclusion

Ce projet démontre comment la pile Bluetooth Classic intégrée à l'ESP32, combinée à la bibliothèque *PS4-esp32*, permet de réaliser un système de contrôle de robot sans fil simple mais efficace. L'abstraction du pilote pont H maintient le firmware clair et facile à étendre. La plateforme constitue une base solide pour des applications robotiques tout-terrain plus avancées, autonomes ou semi-autonomes.

Références

- [1] aed3, *Bibliothèque PS4-esp32*, GitHub. <https://github.com/aed3/PS4-esp32>
- [2] Espressif Systems, *Manuel de référence technique ESP32*, 2023. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [3] STMicroelectronics, *Fiche technique L298N Double Pont H Intégral*, 2000.